Neyshabur, B. Towards learning convolutions from scratch. In *Advances in Neural Information Processing Systems*, 2020.

Neyshabur, B., Tomioka, R., and Srebro, N. In search of the real inductive bias: On the role of implicit regularization in deep learning. *arXiv preprint arXiv:1412.6614*, 2014.

Nguyen, T., Raghu, M., and Kornblith, S. Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth. In *International Conference on Learning Representations*, 2021.

Ott, J., Linstead, E., LaHaye, N., and Baldi, P. Learning in the machine: To share or not to share? *Neural Networks*, 126:235–249, 2020.

Pellegrini, F. and Biroli, G. Neural network pruning denoises the features and makes local connectivity emerge in visual tasks. In *International Conference on Machine Learning*, pp. 17601–17626. PMLR, 2022.

Rosenblatt, F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.

Sahiner, A., Ergen, T., Ozturkler, B., Pauly, J., Mardani, M., and Pilanci, M. Unraveling attention via convex duality: Analysis and interpretations of Vision Transformers. In *International Conference on Machine Learning*, pp. 19050–19088. PMLR, 2022.

Saxe, A., McClelland, J., and Ganguli, S. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *Proceedings of the International Conference on Learning Representations*, 2014.

Schmidhuber, J. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.

Tolstikhin, I., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Keysers, D., Uszkoreit, J., Lucic, M., et al. MLP-Mixer: An all-MLP architecture for vision. In *Advances in Neural Information Processing Systems*, 2021.

Touvron, H., Bojanowski, P., Caron, M., Cord, M., El-Nouby, A., Grave, E., Izacard, G., Joulin, A., Synnaeve, G., Verbeek, J., et al. ResMLP: Feedforward networks for image classification with data-efficient training. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

Wightman, R. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.

Woodworth, B., Gunasekar, S., Lee, J. D., Moroshko, E., Savarese, P., Golan, I., Soudry, D., and Srebro, N. Kernel and rich regimes in overparametrized models. In *Conference on Learning Theory*, pp. 3635–3673. PMLR, 2020.

Yasuda, T., Bateni, M., Chen, L., Fahrbach, M., Fu, G., and Mirrokni, V. Sequential attention for feature selection. In *International Conference on Learning Representations*, 2023.

Yu, W., Luo, M., Zhou, P., Si, C., Zhou, Y., Wang, X., Feng, J., and Yan, S. MetaFormer is actually what you need for vision. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pp. 10819–10829, 2022.

Zhang, A., Tay, Y., Zhang, S., Chan, A., Luu, A. T., Hui, S., and Fu, J. Beyond fully-connected layers with quaternions: Parameterization of hypercomplex multiplications with $1/n$ parameters. In *International Conference on Learning Representations*, 2021.

Zhou, S., Wu, J.-N., Wu, Y., and Zhou, X. Exploiting local structures with the Kronecker layer in convolutional networks. *arXiv:1512.09194*, 2015.

A. Details of Architectures

Here, we overview more technical details of all models: MLP-Mixer, Simple Mixer (S-Mixer), and MLP with sparse weights (SW-MLP). In Section A.1, we introduce the transformation from the input image to the first hidden layer. In Section A.2, we overview some detailed formulation of the models including skip connection and layer normalization.

Table S.1 . Summary table of models. The notation hidden in the block means whether the model has a hidden layer in each MLP block. The symbol (*) indicates the setting depends on the kind of experiment. Here, GAP is the global average pooling.

Model	Per-patch FC	Skip-conn.	Layer-norm	Hidden in the block	GAP
SW-MLP	✓	✓	✓	(*)	✓
(RP) MLP-Mixer	✓	✓	✓	✓	✓
(RP) S-Mixer	✓	✓	✓		✓
Shallow Kronecker					
Shallow Monarch					

A.1. Per-patch FC Layer

The first layer of the MLP-Mixer is given by the so-called per-patch FC layer, which is a single-layer channel mixing. In all experiments, for a patch size P, the input image is decomposed into HW/P^2 non-overlapping image patches with size $P \times P$; we rearrange the $H \times W$ input images with 3 channels into a matrix whose size is given by $(HW/P^2) \times 3P^2 = S_0 \times C_0$. For the rearranged image $X \in \mathbb{R}^{S_0 \times C_0}$, the per-patch fully connected (FC) layer is given by

$$Y = XW^\top, \tag{S.1}$$

where W is a $C \times C_0$ weight matrix. We use the per-patch FC layer not only for Mixers but also for SW-MLP.

Remark on per-patch FC layer: The original study set the size of the mixing layers to $S = S_0$. In contrast, to investigate the contribution of each input image size and hidden layer size independently, it is rational to change (S, C) independent of (S_0, C_0). Therefore, we make the per-patch FC transform the input size C_0 to the output size C and the first token mixing layer transform S_0 to S.

A.2. MLP-Mixer and S-Mixer

Let us denote a block of the MLP-Mixer by

$$f_{W_1,W_2}(X) = \phi(XW_1^\top)W_2^\top, \tag{S.2}$$

and that of the S-Mixer by

$$f_{W_1}(X) = \phi(XW_1^\top). \tag{S.3}$$

We set $\phi = \mathrm{GELU}$.

A.2.1. MLP-Mixer

We set the layer normalization (LN) by

$$\mathrm{LN}(X) = \frac{X - m(X)}{\sqrt{v(X) + \epsilon}} \odot \gamma + \beta, \quad X \in \mathbb{R}^{S \times C}, \tag{S.4}$$

where $\odot$ denotes the Hadamard product , $m(X)$ (resp. $v(X)$) is the empirical mean (resp. the empirical variance) of X with respect to the channel axis, and γ, β are trainable parameters. We set $\epsilon = 10^{-5}$ in all experiments.

In the implementation of fully connected layers, we use only the right matrix multiplications in the same way as the original MLP-Mixer (Tolstikhin et al., 2021). A token-mixing block $X \mapsto U$ of MLP-Mixer is given by

$$U = X + f_{W_1,W_2}(\mathrm{LN}(X)^\top)^\top, \tag{S.5}$$

where W_1 is $S \times \gamma S$ and W_2 is $\gamma S \times S$. Similarly, we set a channel-mixing block $U \mapsto Y$ as

$$Y = U + f_{W_3,W_4}(\mathrm{LN}(U)), \tag{S.6}$$

where W_3, W_4 are weight matrices.

We refer to the composed function $X \mapsto Y$ of the token-mixing block and the channel-mixing one as *a base block* of the MLP-Mixer. The MLP-Mixer with L-blocks is composed in the order of the per-patch FC layer, the L base blocks, and the global average pooling with the layer normalization, and the last fully connected classification layer.

A.2.2. S-MIXER

The S-Mixer without random permutations is implemented by replacing the MLP-block f_{W_1,W_2} and f_{W_3,W_4} in the MLP-Mixer with FC blocks. That is, token-mixing and channel-mixing blocks are given by

$$U = X + f_W(\mathrm{LN}(X)^\top)^\top, \tag{S.7}$$

$$Y = U + f_V(\mathrm{LN}(U)), \tag{S.8}$$

where W and V are weight matrices. The transpose of the input matrix in the token-mixing block is implemented by rearrangement of entries. We decided to apply both skip-connection and layer normalization even in the S-Mixer. This is a rather technical requirement for ensuring the decrease of training loss in deep architectures.

A.2.3. MIXERS WITH PERMUTED KRONECKER LAYERS

Here we implement generalized MLP-Mixer and S-Mixer with permutation matrices and PK-layers. Recall that for any matrix X,

$$X^\top = \mathrm{Mat}(J_c \mathrm{vec}(X)), \tag{S.9}$$

where J_c is the $m \times m$ commutation matrix. Therefore, the token-mixing block of the S-Mixer is

$$U = X + \mathrm{Mat} \circ J_c^\top \circ \mathrm{vec} \circ f_W \circ \mathrm{Mat} \circ J_c \circ \mathrm{vec} \circ \mathrm{LN}(X) \tag{S.10}$$

$$= X + \mathrm{Mat} \circ \text{PK-Layer}_W(\mathrm{LN}(X); J_c, J_c^\top). \tag{S.11}$$

Similarly, the channel-mixing block of the S-Mixer is equal to

$$Y = U + \mathrm{Mat} \circ \text{PK-Layer}_V(\mathrm{LN}(U); I, I), \tag{S.12}$$

where I is the identity matrix. Note that skip-connections gather J_c and $J_c^\top$ in the same mixing block for compatibility in shapes of hidden units.

To get examples of PK family and to generalize Mixers, we implement the random permuted (RP) S-Mixer with skip-connections by replacing J_c and $J_c^\top$ with i.i.d. random permutation matrices J_1 and J_2:

$$U = X + \mathrm{Mat} \circ \text{PK-Layer}_W(\mathrm{LN}(X); J_1, J_2). \tag{S.13}$$

We implement the random permutations by random shuffling of output $m = SC$ indexes of vectors. We freeze it during the training step. Note that we avoid using an $m \times m$ matrix representation of J_x for memory efficiency. We implement the random permuted (RP) MLP-Mixer by the same way as the RP-S-Mixer.

A.2.4. THE SKIP-CONNECTION IN THE FIRST BLOCK

The first token-mixing block has the input shape (S_0, C) and the output shape (S, C). However, we need to change S with fixing S_0 in some experiments. To control the difference of S_0 and S, we set the first token-mixing block as follows:

$$U = \text{SkipLayer}(X) + \text{PK-Layer}_W(\mathrm{LN}(X); J_1, J_2), \tag{S.14}$$

where the skip layer is given by

$$\text{SkipLayer}(X) = \mathrm{LN}(\tilde{W} X), \tag{S.15}$$

where $\tilde{W}$ is a $S \times S_0$ weight matrix. For a fair comparison, we use the skip layer even if $S = S_0$ in the experiments that we sweep S. We use the same setting for the MLP-Mixer as for the S-Mixer.